AI Interview Coach — Complete Project Documentation

Document Created: February 2026
Project Type: AI-Powered Mock Interview & Coding Assessment Platform

---

1. Project Overview

AI Interview Coach is a web-based interview practice platform that uses AI (Ollama / LLaMA) to simulate realistic technical and behavioral interviews.

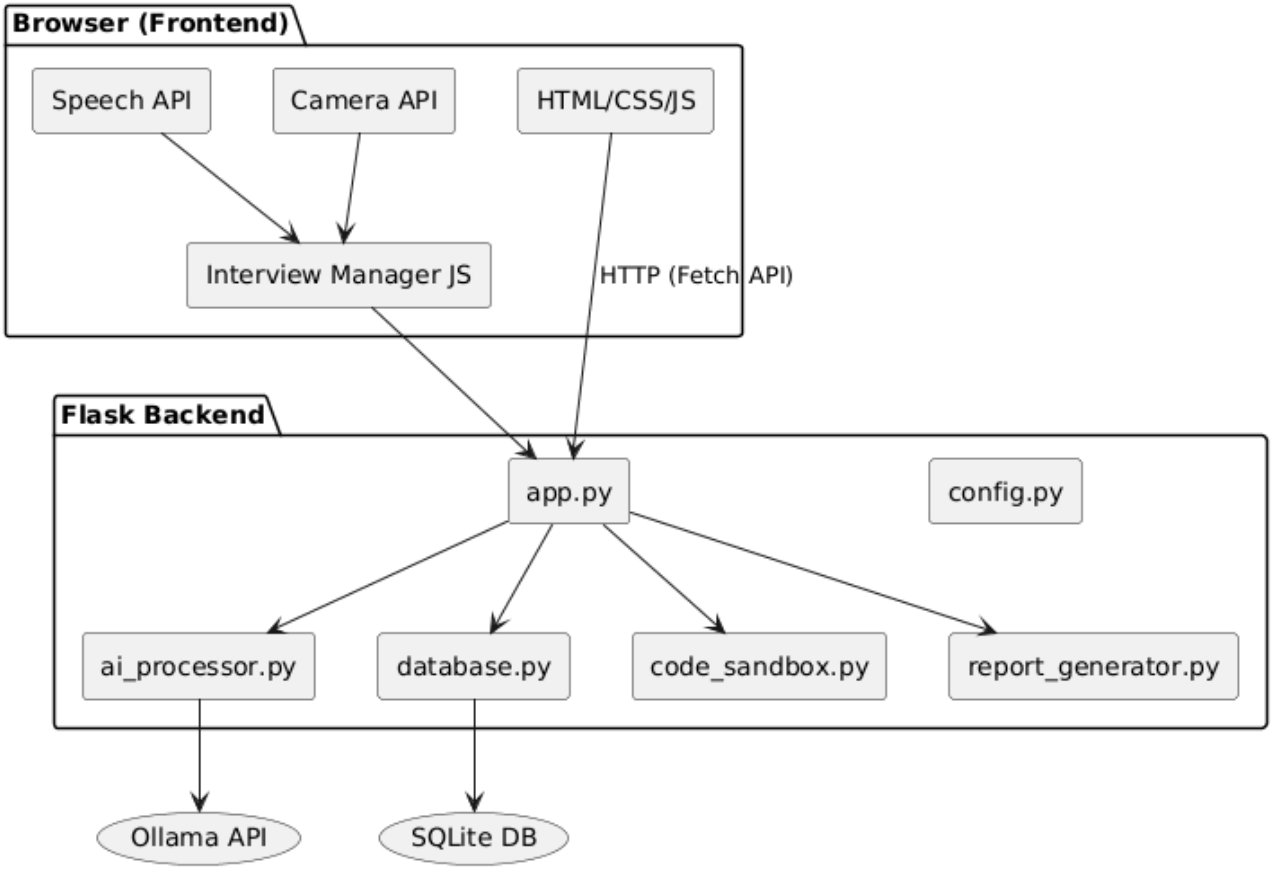
Users upload a resume and job description, then face AI-generated questions tailored to their profile. The system evaluates answers in real time, conducts a coding test, and produces a comprehensive performance report.

Key Capabilities

- Resume-aware question generation (based on resume + job description)
- Browser-based speech recognition (Web Speech API)
- Real-time scoring (Grammar, Relevance, STAR, Confidence)
- Secure Python coding sandbox
- AI-powered evaluation (LLaMA 3.2 via Ollama)
- Final PDF performance report generation

2. System Architecture

High-Level Architecture Diagram



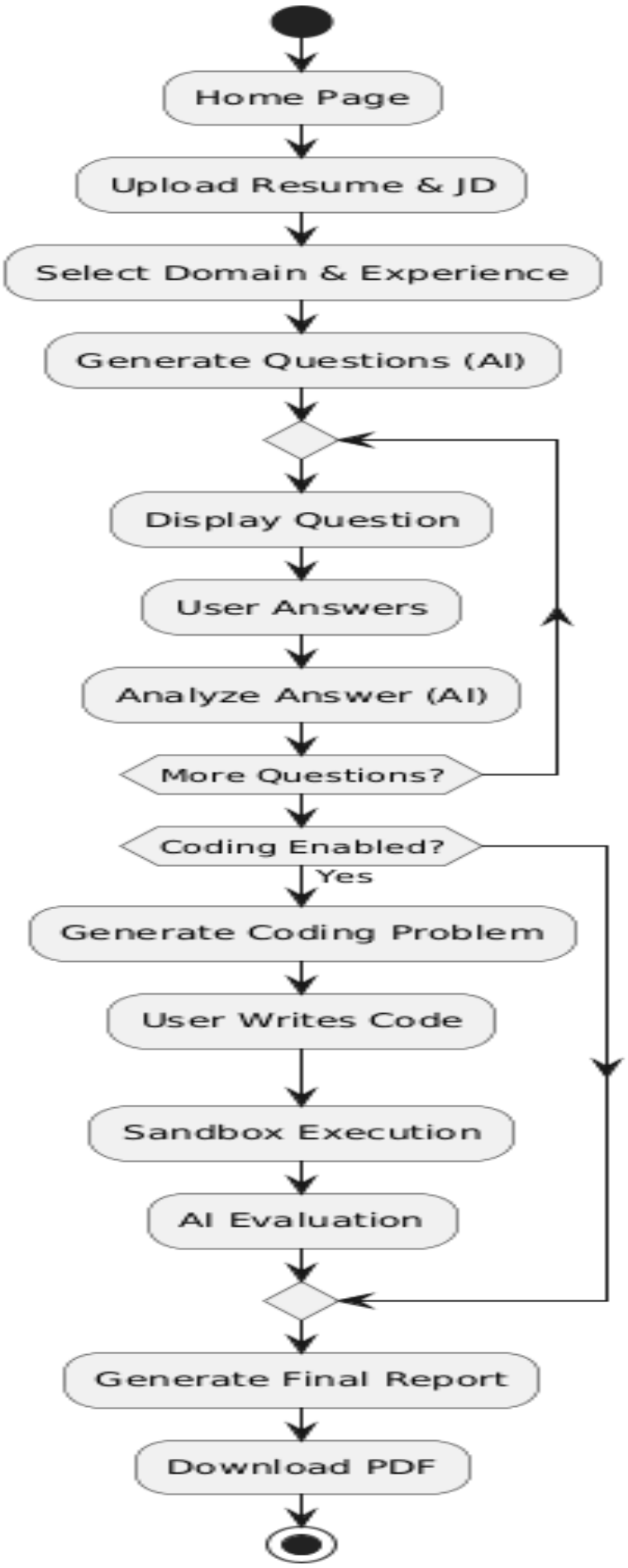
Data Flow Summary

1. Resume uploaded → Text extracted via PyPDF2 → Stored in Flask session
2. Domain + experience selected → Stored in session
3. AI generates 8 questions → Saved in DB + session
4. User answers → AI scoring → Saved to DB
5. Coding test → Sandbox execution → AI evaluation
6. Final report → Generated + Displayed + PDF Download

3. Technical Stack

Layer	Technology	Purpose
Frontend	HTML5, Tailwind CSS, JavaScript	UI Rendering
Icons	Font Awesome 6.4	UI Icons
Templating	Jinja2	Server-side templates
Backend	Flask (Python)	Routing & Session Management
AI Engine	Ollama + LLaMA 3.2:1b	Question + Evaluation Engine
NLP	TextBlob, NLTK	Sentiment + Tokenization
Database	SQLite3	Persistent Storage
Sessions	Flask-Session	Server-side sessions
File Parsing	PyPDF2	Resume extraction
PDF Reports	pdfkit / xhtml2pdf	Report generation
Speech	Web Speech API	Speech recognition
Camera	MediaDevices API	Webcam preview

4. Complete User Workflow

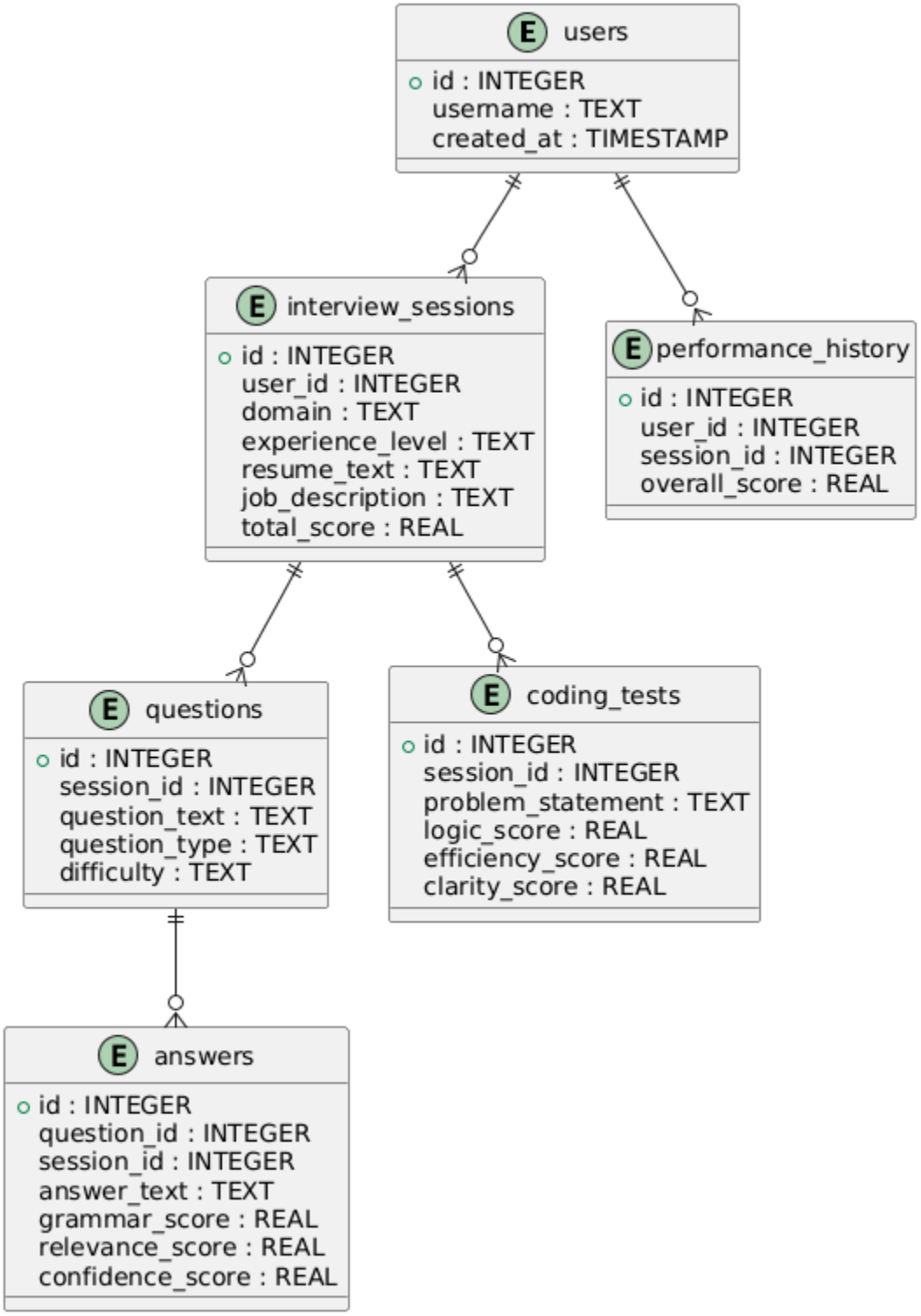


Step-by-Step Flow

1. Home → Start Practice Session
2. Upload → Resume + JD stored
3. Setup → Domain + Level selected
4. Interview → 8 AI questions + scoring
5. Coding → Python challenge + evaluation
6. Feedback → Summary scores
7. Report → Final AI analysis + PDF

5. Database Schema

Database Architecture Diagram



Tables

- users
- interview_sessions
- questions
- answers
- coding_tests
- performance_history

Each table contains structured relational data linking sessions, questions, answers, coding results, and performance history.

---

6. API Architecture

Main Routes

Method	Route	Description
GET	/	Home page
GET/POST	/upload	Resume upload
GET/POST	/setup	Interview config
GET	/start-interview	Generate questions
GET	/coding-test	Coding problem
GET	/feedback	Score summary
GET	/generate-report	Final AI report
GET	/download-report	PDF download

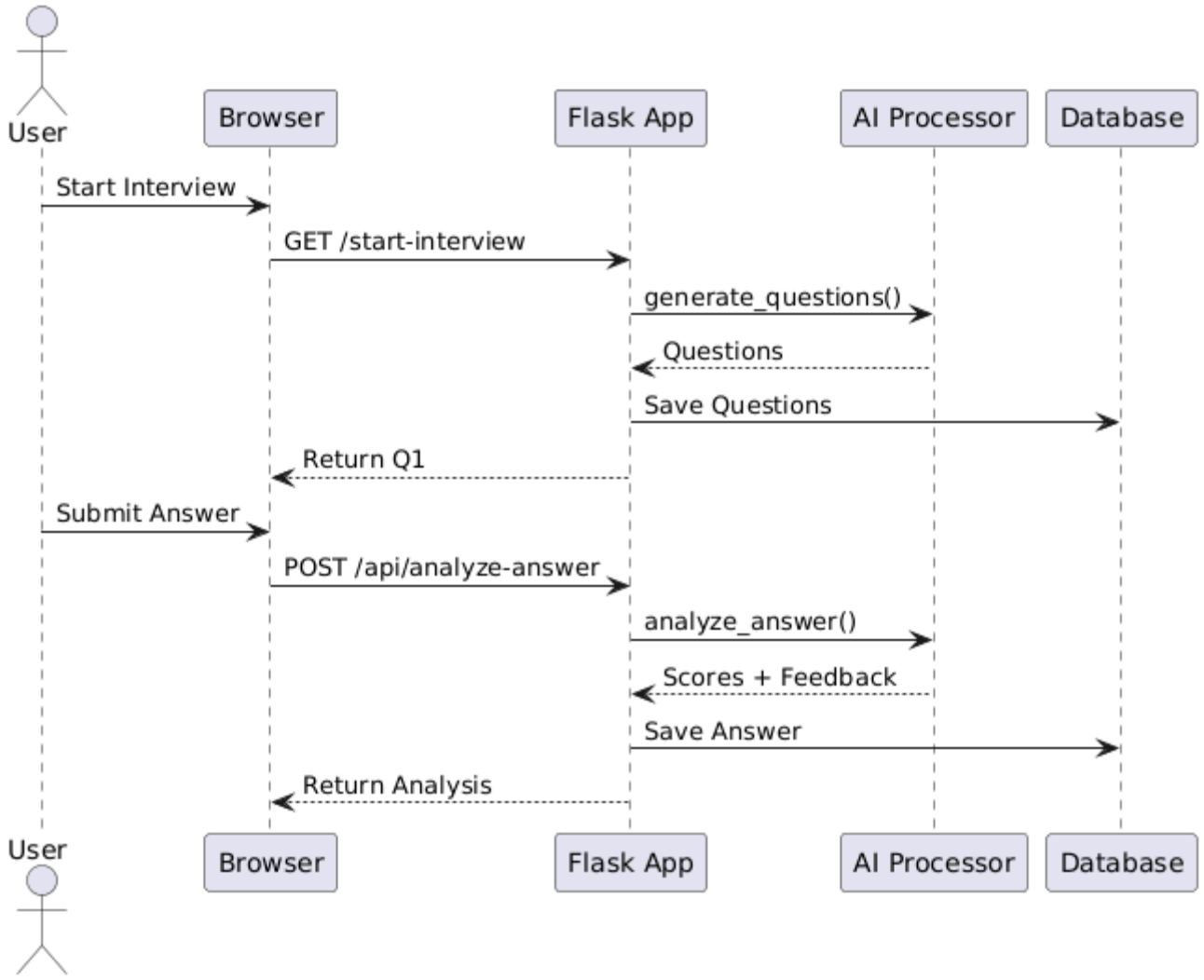
Core API Endpoints

- POST /api/next-question
- POST /api/analyze-answer
- POST /api/evaluate-code
- POST /api/speech-status

All endpoints use JSON request/response format.

---

7. Interview Flow (Detailed)

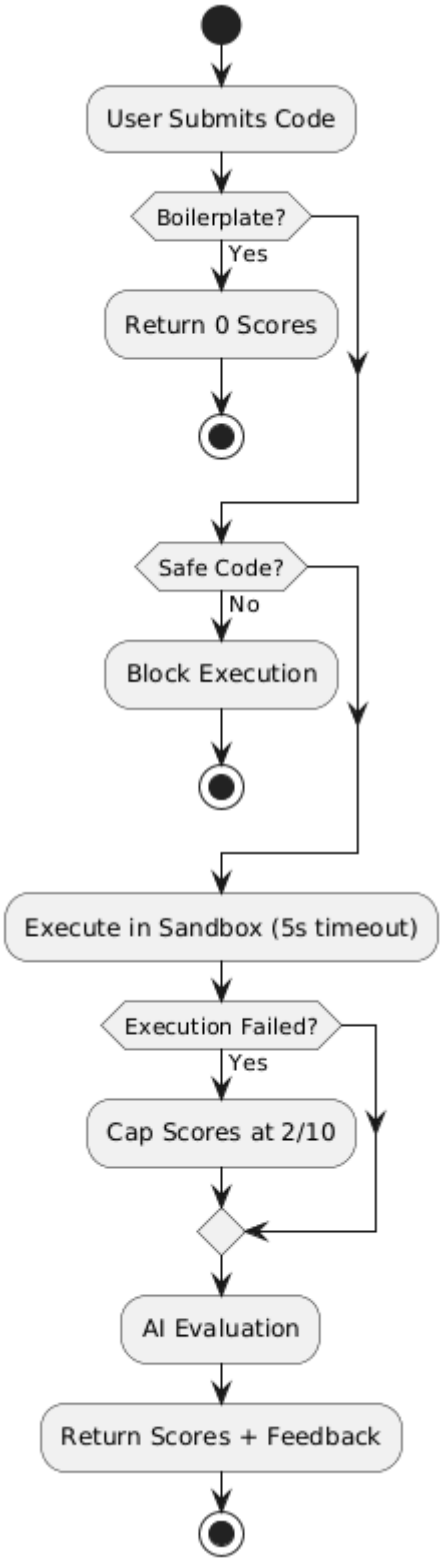


Coding Test / Feedback

Features:

- 2-minute timer per question
- Speech-to-text input
- Confidence scoring
- Cross-question generation
- Webcam preview

8. Coding Test Flow



Security Layers

- Pattern blocking (os, sys, eval, subprocess, socket, etc.)
- Timeout protection
- PYTHONPATH cleared
- Boilerplate detection

9. Scoring Logic

Interview Scoring (0–10)

- Grammar
- Relevance
- STAR Method
- Confidence (Composite)

Confidence Formula

Confidence =

(0.30 × Relevance)

+ (0.20 × STAR)

+ (0.20 × Sentiment)

+ (0.15 × Filler Penalty)

+ (0.15 × Length Score)

Filler words detected:

um, uh, ah, er, like, you know, so, well

Coding Scores

- Logic (0–10)
- Efficiency (0–10)
- Clarity (0–10)
- Test Cases (0–5)
- Time Complexity
- Space Complexity

10. Module Overview

app.py

Main controller (436 lines)

ai_processor.py

AI integration & scoring engine (425 lines)

database.py

SQLite data layer (284 lines)

code_sandbox.py

Secure execution layer (131 lines)

report_generator.py

HTML + PDF report system (587 lines)

config.py

Central configuration (25 lines)

---

11. Security Considerations

- Sandboxed execution
- File upload validation
- Server-side sessions
- Input auto-escaping (Jinja2)
- Timeout protections

Limitations:

- No authentication
- No CSRF protection
- Basic sandbox (not Docker)

---

12. Error Handling

- Ollama failure → Fallback questions
- AI JSON parsing fallback strategies
- Timeout handling
- Empty input prevention
- Speech API compatibility checks

---

13. Edge Cases Handled

- Empty answer
- Boilerplate code
- Syntax errors
- Infinite loops
- Speech permission denied
- AI unavailable

---

14. Known Limitations

1. Single user mode
2. Python-only coding
3. AI-estimated test cases
4. Basic sandbox security
5. Ollama dependency
6. No session persistence across restart
7. English-only UI

15. Future Improvements

- Authentication system
- Multi-language coding
- Docker sandbox
- Real test case engine
- Analytics dashboard
- Video recording
- WebSocket streaming
- Mobile responsiveness

---

16. Setup & Installation

Prerequisites

- Python 3.9+
- Ollama installed
- (Optional) wkhtmltopdf

Installation Steps

```
git clone <repository-url>
cd AI-Interview-Platform
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
ollama pull llama3.2:1b
ollama serve
flask run
```

Open:

http://127.0.0.1:5000

17. Project File Structure

```
AI-Interview-Platform/
├── app.py
├── ai_processor.py
├── code_sandbox.py
├── config.py
├── database.py
├── report_generator.py
├── requirements.txt
├── database.sqlite
├── templates/
├── static/
├── uploads/
└── flask_session/
```

Final Summary

AI Interview Coach is a fully integrated AI-powered interview and coding assessment system combining:

- Resume-aware intelligent questioning
- Real-time NLP-based scoring
- Secure coding sandbox execution
- AI-generated performance reports

It simulates realistic interview environments and provides structured, measurable feedback for candidate improvement.

End of Document.